ChessCam – gjenkjenning av et fysisk sjakkbrett

Marius Phillips
DAT158 – Prosjektoppgave 2, høsten 2026

1. Problemstilling og løsning

Målet med ChessCam er å registrere et sjakkparti med kameraet på en mobiltelefon. Spilleren bruker et vanlig brett, mens appen leser stillingen og lagrer trekkene digitalt. Partiet kan deretter analyseres uten at spilleren først må skrive inn alle trekkene.

Oppgaven for maskinlæringen er å finne brikkene i kamerabildet og bestemme farge og brikketype. Dette er vanskeligere enn å kjenne igjen én brikke på et ryddig bilde. Kameraet ser brettet på skrå, brikker skjuler hverandre, og en hånd kan dekke brettet. Lys og ulike brikkesett gir også variasjon.

ChessCam bygger videre på CameraChess av Pbatch [1]. Den bruker LeYOLO-arkitekturen [2] og eksisterende modellvekter som er finjustert med flere bilder. Arkitekturen og grunnmodellen er derfor hentet fra tidligere arbeid. Arbeidet som beskrives her, gjelder tilpasning av data, videre trening, evaluering og bruk av modellene i produktet.

Etter avtale med Steffen Mæland 28. oktober 2026 leveres treningskode og en innspilt demo uten hele appens kildekode. Nettside og videolenke samles i README-filen.

2. Hvordan maskinlæringen brukes

Løsningen bruker veiledet læring. Treningsbildene har en fasit med bokser rundt brikkene og navn på klassene. Det er tolv brikkeklasser: bonde, springer, løper, tårn, dronning og konge, hver i svart og hvitt.

Et konvolusjonsnettverk bruker filtre som lærer mønstre i bilder. Tidlige lag kan finne enkle trekk som kanter, mens senere lag kombinerer dem til mer sammensatte former [3]. LeYOLO er en lett objektdetektor som passer til en telefon med begrenset regnekraft. Modellen foreslår både bokser og klasseskårer for brikkene.

Finjustering er en form for overføringslæring: treningen starter med vekter som allerede har lært å kjenne igjen sjakkbrikker. Vektene tilpasses deretter nye bilder. Dette krever mindre ny læring enn å starte fra tilfeldige vekter [4]. En risiko er overtilpasning, der modellen blir god på treningsmaterialet, men dårlig på nye brett.

Appen har også en modell som finner de 49 indre krysspunktene i brettet. Geometri brukes til å koble brikkene til ruter. Deretter vurderer vanlig programkode flere bilder og mulige lovlige trekk. En enkelt brikkedeteksjon er altså ikke nok til å registrere et trekk.

Modellene kjøres i nettleseren med TensorFlow.js. Bildet skaleres med bevart sideforhold og fylles ut til 288 × 480 piksler. Pikselverdiene deles på 255 før modellen brukes.

3. Data og klargjøring

Den siste dokumenterte treningsrunden for brikkemodellen heter `real-v2`. Den brukte følgende bilder før vekting gjennom gjentakelser:

Datakilde	Trening	Validering
Opprinnelig CameraChess-materiale	35 266	72
Prosjektets tilleggsbilder	134	44
Første eksterne tillegg	105	18
Andre eksterne tillegg	1 897	192
Sum	**37 402**	**326**

Det første eksterne tillegget bruker datasettene til Nina Daithal og Tanner Gilbert. Det andre bruker Fenit og datasettet publisert av surawut. Kildene er oppgitt nedenfor. De egne tilleggsbildene har også 47 separate testbilder; disse inngår ikke i treningen eller valideringstabellen over.

Klargjøringen samordner klassenumrene mellom datasettene og konverterer merkingen til YOLO-format. Hver brikke beskrives med klasse, boksens midtpunkt, bredde og høyde. Koordinatene er relative til bildestørrelsen. Klassen «Hand» fjernes fra materialet som inneholder denne, siden oppgaven her er å finne sjakkbrikker.

Identiske og svært like bilder kontrolleres for å redusere dubletter. De egne bildene deles etter opptaksøkt. De nye datakildene i `real-v2` beholdt likevel i hovedsak sine opprinnelige trenings- og valideringsinndelinger etter klargjøringen. Nærliggende bilder fra samme video kan derfor fortsatt finnes på begge sider. Det kan gi for gode valideringsresultater sammenlignet med et helt nytt brettoppsett.

4. Trening

`real-v2` startet fra prosjektets tidligere `external-v1`-modell. Treningen brukte LeYOLO-forkens Ultralytics 8.1.4 og PyTorch 2.2.2 på en NVIDIA L4. Den lagrede loggen viser at runden tok omtrent 1 time og 59 minutter, inkludert forberedelser og validering.

De viktigste innstillingene var maksimalt 20 epoker, batchstørrelse 64, startlæringsrate 0,00005 og AdamW som optimizer. AdamW er metoden som oppdaterer vektene, med separat vektnedbrytning som regularisering [5]. Tidlig stopp hadde tålmodighet 6, slik at treningen kunne avsluttes ved manglende forbedring etter trenerens kriterium.

Under trening varieres blant annet lys, farge, skala og perspektiv. Horisontal speiling brukes også, og boksmerkingen endres sammen med bildet. Dette gir variasjon uten å samle inn et nytt bilde for hver variant.

De små datakildene ville ellers hatt liten innflytelse sammenlignet med det opprinnelige materialet. Derfor gjentas de egne treningsbildene 15 ganger og de eksterne treningsbildene fire ganger. Gjentakelsene gir kildene større vekt, men skaper ikke flere uavhengige bilder. Enkelte valideringskilder gjentas også ved valg av beste modell. Til slutt valideres hver kilde separat uten disse gjentakelsene.

`train.py` er et forenklet utdrag av denne treningen. Det beholder modellinnstillinger, sammensetting av klargjorte datasett, vekting, finjustering og separat validering. Skylagring og behandling av råarkiver er utelatt. Koden forutsetter derfor ferdig merkede data og kompatible startvekter.

5. Resultater og vurdering

En objektdetektor må finne riktig klasse og plassere boksen riktig. **Precision** sier hvor stor andel av funnene som er riktige, mens **recall** sier hvor stor andel av fasitobjektene modellen finner.

IoU måler hvor mye modellens boks overlapper fasitboksen. **mAP50** oppsummerer presisjon og gjenfinning over skårterskler og klasser ved IoU-grense 0,50. **mAP50–95** bruker flere grenser fra 0,50 til 0,95 og stiller dermed strengere krav til boksplasseringen [6].

Tabellen viser lagrede valideringsresultater fra `real-v2`, hentet fra treningsloggen og gjengitt i `results.json`. Ultralytics-valideringen brukte bildestørrelsesinnstilling 480. Tallene er avrundet til fire desimaler.

Valideringskilde	mAP50	mAP50–95	Precision	Recall
Opprinnelig materiale	0,9854	0,7972	0,9617	0,9625
Egne tilleggsbilder	0,9625	0,8362	0,9337	0,9020
Første eksterne tillegg	0,8861	0,6107	0,8302	0,8539
Andre eksterne tillegg	0,9923	0,9444	0,9955	0,9771

Resultatene varierer mellom kildene. Modellen gjør det bedre på det andre eksterne tillegget enn på det første. Dette kan skyldes forskjeller i vanskelighetsgrad, bildelikhet og hvilke brett treningsmaterialet dekker. Tallene

alene avgjør ikke hvilken forklaring som er viktigst. Mulig likhet mellom trenings- og valideringsbilder må særlig tas med i vurderingen av den høyeste skåren.

Disse tallene måler brikkedeteksjon i treningsoppsettets validering. De viser ikke hvor ofte appen registrerer et helt sjakkparti riktig. En mAP på 0,99 betyr ikke 99 prosent riktige trekk. Det finnes ikke en lagret uavhengig måling av siste modell gjennom hele appens bildebehandling og trekkregistrering.

Neste steg bør være å teste hele partier med kjente trekk og helt nye brettoppsett. Da kan man måle riktige trekk, feilregistreringer og forsinkelse. En ny trening bør også holde alle bilder fra samme opptak samlet i én del av datasettet. Det vil gi et bedre grunnlag for å vurdere om modellen fungerer utenfor de kjente bildene.

6. Kilder og bruk av KI

[1] Pbatch: CameraChessWeb. Utgangspunkt for appen, modellvektene og det opprinnelige bildematerialet.

[2] Hollard, L. mfl.: LeYOLO, New Embedded Architecture for Object Detection, 2024/2025. Implementasjon, commit `b1882a118da127a9bb299c8e906d23ed422cb972`.

[3] Stanford CS231n: Convolutional Neural Networks.

[4] TensorFlow: Transfer learning and fine-tuning.

[5] Loshchilov, I. og Hutter, F.: Decoupled Weight Decay Regularization, ICLR 2019.

[6] COCO: Evalueringskode for objektdeteksjon.

Eksterne treningsdata:

• Nina Daithal: Chess Pieces Dataset, med oppgitt Roboflow-kilde. CC BY 4.0 etter kildemetadata. Klasser samordnes og dubletter fjernes.

• Tanner Gilbert: Chess Piece Detection. CC BY 4.0. Merkingen konverteres fra Pascal VOC til YOLO, og dubletter fjernes.

• Theo Harjanto, Vincent Oei og Dicky Susanto: Annotated Image Dataset for Fenit, versjon 1. CC BY 4.0. Klasser samordnes og en identisk kopi fjernes.

• surawut: Chess Pieces Detection Dataset, revisjon `764dfdfdacbce90c4a325b37cc7bfb91dff1e799`. CC BY 4.0. Bearbeidede bildevarianter reduseres, «Hand» fjernes og klasser samordnes.

CC BY 4.0 gjelder de fire eksterne kildene over. Det er ikke avklart en egen uttrykkelig lisens for Pbatchs publiserte modellvekter og bildearkiver. Bilder og vekter er ikke inkludert i kodeutdraget.

OpenAI Codex ble brukt til å hente ut og forenkle kode, utarbeide rapportutkast, finne kilder og kontrollere innleveringsfilene. Resultattallene kommer fra den eksisterende treningsloggen; det er ikke gjennomført en ny trening som del av dette arbeidet.